

#3 Real-Time HDR Tone Mapping



1. Project Overview

The objective of this project was to implement an HDR (High Dynamic Range) Tone Mapping algorithm using NVIDIA CUDA technology to accelerate image processing. The project involved creating a framework for HDR filter selection via command-line arguments, implementing the mathematical model for luminance transformation, and optimizing GPU-based computations.

Note on Environment: Due to the use of the Google Colab environment, processing a live webcam stream was adapted to static images, allowing for accurate performance benchmarking between CPU and GPU implementations on identical data.

2. Implementation (Core Implementation)

As part of the framework extension, the following steps were performed:

- Added a new HDR_TONEMAPPING filter type to the FilterType enum.
- Implemented parsing logic for HDR parameters in FilterUtils.
- Created the hdrToneMappingKernel CUDA kernel, which performs:
 - Normalization of RGB channels.
 - Application of the exposure coefficient and range compression operator: $\text{color} = \text{color} / (1 + \text{color})$.
 - Luminance and saturation adjustment.
 - Gamma correction ($1/\text{gamma}$).

3. Performance and Optimization

Memory Transfer Optimization

The primary bottleneck in GPU computing is the data transfer between the Host (CPU) and the Device (GPU).

The Problem: Initially, GPU memory allocation via cudaMalloc occurred during every frame processing cycle, creating a significant latency.

The Solution: A dedicated HDRProcessor class was implemented to initialize GPU memory once at application startup.

Result: Eliminating system memory calls within the main processing loop improved execution time stability.

Shared Memory Analysis

The use of __shared__ memory was deemed *inexpedient*.

The Global Tone Mapping algorithm is pixel-independent (each thread calculates its own pixel value without accessing neighbors). Shared memory is highly effective for convolution-based algorithms (e.g., blurring, edge detection) where threads share input data. In this implementation, forcing data into Shared Memory would have introduced unnecessary read/write operations, decreasing performance rather than increasing it.

4. Performance Comparison: GPU vs. CPU

During testing, the CPU implementation demonstrated lower processing times compared to the GPU.

This is attributed to the following factors:

1. *Task Scale:* The Tone Mapping algorithm used is computationally "light." Modern CPUs are fast enough to process the data array before the GPU can complete the overhead of kernel launching and PCIe data synchronization.
2. *PCIe Latency:* The physical time required to transfer data between the CPU and GPU via the PCIe bus becomes the dominant factor when the computational workload is relatively small.
3. *Conclusion:* The GPU demonstrates its true efficiency in tasks where computational complexity (FLOPs) significantly outweighs data transfer overhead. For simple per-pixel operations, the CPU remains the more efficient tool.